

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026



Project Name: StockFlowAI

Version: 1.0

Domain: 1

Team Name: Jet2Holiday

Team Members: HIEW YI HWA

Repository URL:

Prepared For: UMHackathon 2026 Preliminary Round

System Title: **AI Store Operations Copilot for Stock Directory and Supplier Invoice Processing**

Table of Contents

1.0	Introduction.....	3
1.1	Purpose	3
1.2	Background	4
2.0	System Architecture & Design	6
3.0	Functional Requirements & Scope	11
4.0	Monitor, Evaluation, Assumptions & Dependencies.....	12
5.0	Project Management & Team Contributions.....	14
5.2	Team Members Role.....	14
5.3	Recommendations	14
6.0	Reference	15
6.	Reference	Error! Bookmark not defined.

1.0 Introduction

StockFlow AI is a workflow automation system designed to improve manual retail store operations related to supplier invoice verification and stock discrepancy handling. This document explains the technical design, architecture, workflow logic, and implementation approach of the proposed system. The structure of this document follows the type of sections emphasized in the sample SAD, including architecture, LLM service layer, sequence flow, data flow, MVP scope, NFRs, and external dependencies.

1.1 Purpose

The purpose of this System Analysis Document is to describe the technical scope and design decisions behind **StockFlow AI**, an AI-powered store operations workflow system for stock discrepancy and supplier invoice processing. This document explains how the system is structured, how Z.AI GLM is used as the central reasoning engine, and how the workflow is designed to convert unstructured retail operation inputs into structured, actionable outputs. This is aligned with the hackathon domain requirement for unstructured input understanding, multi-step reasoning, workflow orchestration, structured outputs, and handling of ambiguity or incomplete data.

1. Architecture

This document explains the overall system architecture of StockFlow AI, including the frontend, backend, workflow controller, LLM service layer, validation logic, and database design. It shows how these components work together to support a stateful workflow for stock discrepancy and supplier invoice issue handling.

2. Data Flows

This document explains how discrepancy-related data moves through the system, starting from user input, moving through the AI reasoning layer, then through validation and routing logic, and finally being stored as a structured case record.

3. Model Process

This document describes how **ilmu-glm-5.1** is used as the central reasoning engine in the system through the configured provider endpoint. It explains how the model handles classification, extraction, missing-information detection, and structured workflow output generation. This directly supports the hackathon requirement that GLM be central to workflow reasoning and execution.

4. Role of Reference

This document serves as a technical reference for the development team, QA team, mentors, and judges to understand the overall system design, component interactions, and implementation scope of StockFlow AI.

1.2 Background

In retail receiving operations, supplier invoices and delivered stock are often checked manually. When the invoiced quantity does not match the actual received quantity, staff usually record the issue in notes, spreadsheets, or chat messages, then escalate it informally to managers or finance teams. This results in repeated manual key-in work, inconsistent reporting, slow follow-up, and weak traceability. This type of fragmented and manual workflow directly matches the problem described in the hackathon domain brief.

1. Previous Version

The existing workflow is mostly manual and depends heavily on human interpretation. A typical process is:

1. Store staff compare invoice and delivered stock manually
2. Identify mismatch or issue
3. Type or write the discrepancy in a note, spreadsheet, or chat
4. Supervisor or reviewer interprets the report manually
5. Issue is followed up through repeated communication or re-entry into another system

This process creates delays, inconsistencies, and errors.

1. Changes in Major Architectural Components

StockFlow AI introduces a new AI-centred workflow layer into the manual process. Instead of relying fully on human interpretation, the system adds:

- A discrepancy case intake layer
- An LLM reasoning layer
- A business rules and validation layer
- A routing recommendation layer
- And a case state tracking layer

This transforms a fragmented manual process into a structured workflow engine, which is directly aligned with the domain expectation of multi-step orchestration and adaptive workflow execution.

2. New Capabilities Introduced

- Natural-language discrepancy intake
- AI case classification
- Entity extraction from unstructured input
- Missing information clarification
- Workflow action recommendation
- Structured case generation
- Stateful case tracking

1.3 Target Stakeholder

Stakeholder	Role	Expectation
Store Staff / Receiving Staff	Report invoice and stock discrepancy issues	Quick issue reporting without repeated manual key-in
Store Manager / Supervisor	Review cases and decide next steps	Clear case summary and recommended workflow path
Finance / Procurement Reviewer	Handle invoice disputes or supplier follow-up	Accurate structured discrepancy details
Development Team	Build and maintain the prototype	Modular architecture and clear service boundaries
QA Team	Validate workflow behaviour and AI output quality	Clear expected outputs, edge cases, and test coverage

2.0 System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview

Type	Details
System	Web-based internal operations workflow prototype
Architecture	Client-server workflow system with AI service layer
Frontend	Streamlit
Backend	FastAPI
Database	SQLite
LLM Service	Ilmu-glm-5.1
Development Tool	VS Code

2.1.1.1 Architecture Description

StockFlow AI is organized into five major layers:

1. User Interface Layer

A lightweight frontend used by store staff and reviewers to submit discrepancy descriptions and view structured case results.

2. Workflow Controller Layer

Handles request intake, state transitions, routing decisions, validation calls, and communication between components

3. LLM Service Layer

Uses **ilmu-glm-5.1** through the configured provider endpoint to perform reasoning tasks such as case classification, field extraction, missing-information detection, and structured output generation.

4. Business Rules & Validation Layer

Performs deterministic checks such as required-field validation, quantity comparison, output schema validation, and route recommendation logic

5. Persistence Layer

Stores discrepancy cases, extracted fields, case states, and workflow logs for traceability and later review.

2.1.2 LLM as Service Layer

The LLM layer is the core reasoning engine of StockFlow AI and is implemented using **ilmu-glm-5.1** through the configured provider endpoint <https://api.ilmu.ai/anthropic>. The model is responsible for:

- classifying the discrepancy case type,
- extracting relevant entities from unstructured store input,
- identifying missing information,
- recommending the likely workflow path,
- and returning structured output for downstream processing.

This is important because the sample SAD explicitly requires GLM to be shown as a **service layer**, and the domain brief requires the GLM component to be the central reasoning engine of the workflow. Official GLM-5.1 documentation also shows that the model supports **structured output**, **function calling**, **context caching**, and long-context processing, which are well-suited to agentic workflow systems.

2.1.2.1 Prompt Construction and Context Handling

For each discrepancy case, the backend constructs a prompt that includes:

- the raw user input,
- the expected workflow categories,
- the list of required fields,
- a request for structured JSON-style output,
- and any optional case metadata such as store name or invoice identifier if available.

Only relevant context is sent to the model to control latency and token usage. Official GLM-5.1 documentation states that the model supports **200K context length** and **128K maximum output**, but the system intentionally uses shorter, structured prompts for efficiency and reliability.

2.1.2.2 Response Parsing

The backend parses the LLM response into:

- case type,
- summary,
- extracted fields,
- missing fields,
- confidence / review-needed flags,

- and workflow recommendation.

The parsed response is then validated before being stored or displayed. This directly matches the sample SAD's requirement to show how responses are received, parsed, and passed to the next system component.

2.1.3 Dependency Diagram

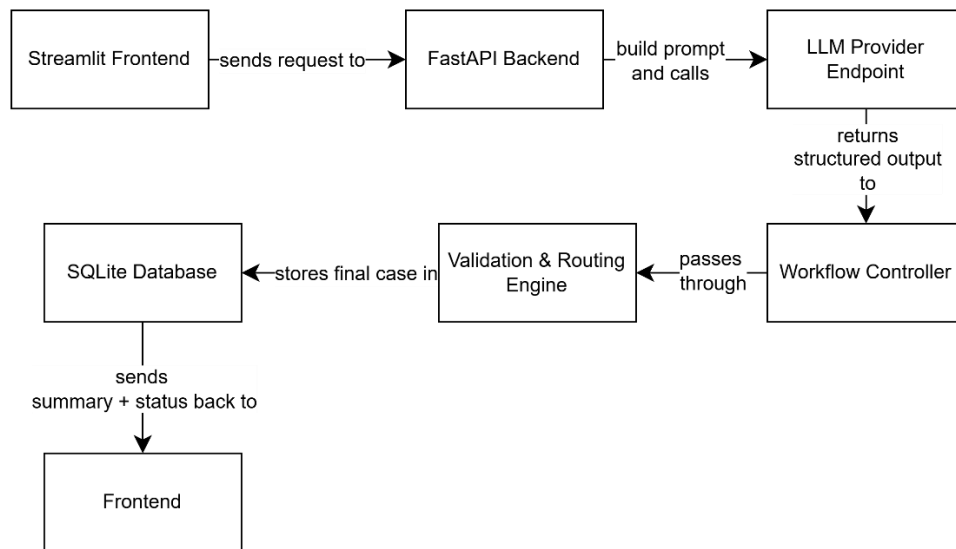


Figure 1: Dependency Diagram

2.1.4 Sequence Diagram

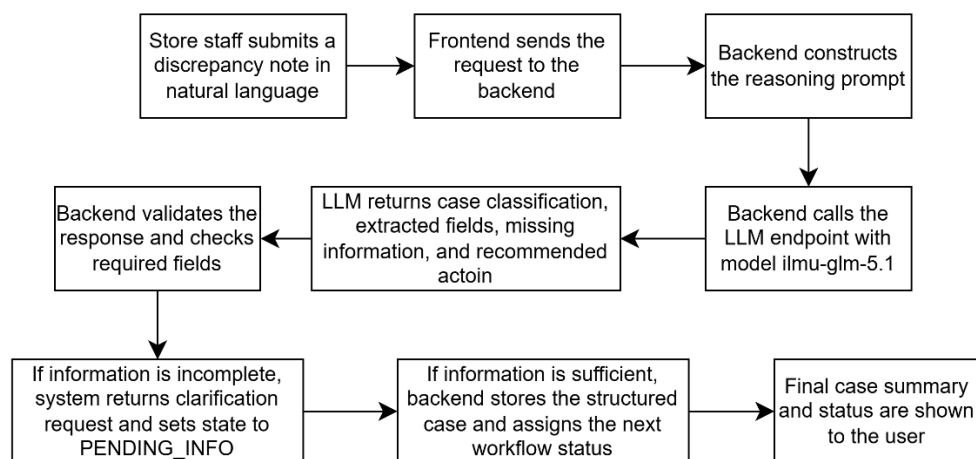


Figure 2: Report Stock Discrepancy Flow

2.2 Technological Stack

Layer	Technology	Justification
Frontend	Streamlit	Fast UI development for demo-ready workflow interaction
Backend	FastAPI	Lightweight API framework with clean request / response handling
Database	SQLite	Simple local persistence suitable for MVP
LLM Integration	Ilmu-glm-5.1 model	Required central reasoning engine for workflow classification, extraction, and orchestration
Language	Python	Fast development for AI workflow systems
Development Tool	VS Code	Efficient local development and debugging

2.3 Key Data Flows

2.3.1 Data Flow Description

The system processes discrepancy data through the following flow:

1. User submits unstructured discrepancy input
2. Backend receives and prepares the input for reasoning
3. LLM interprets the case and returns structured output
4. Backend validates and routes the result
5. Case is stored in the database
6. Final status and summary are returned to the user.

2.3.1.1 Data Flow Diagram (DFD)

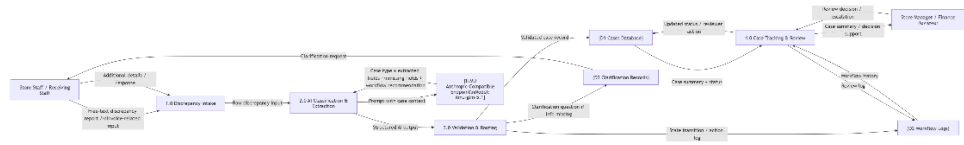


Figure 3: Data Flow Diagram

2.3.2 Normalized Database Schema

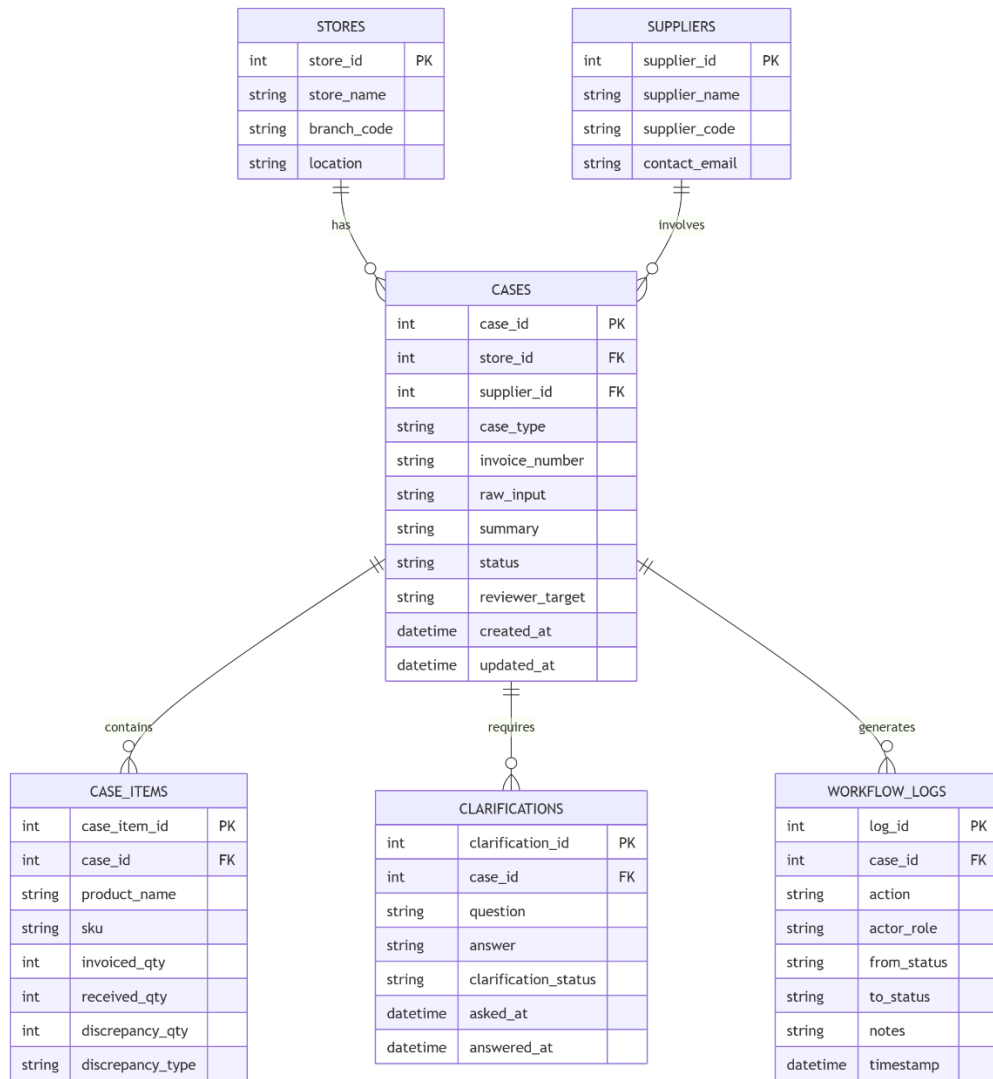


Figure 4: ERD

3.0 Functional Requirements & Scope

2.1 Minimum Viable Product

No.	Feature	Description
1	Discrepancy Intake	User submits discrepancy issue in natural language
2	AI Case Classification & Extraction	LLM classifies the issue and extracts fields
3	Missing-Information Clarification	System detects missing fields and asks follow-up questions
4	Workflow Recommendation	System recommends manager review, finance review, or dispute handling
5	Case State Tracking	Case state is stored and displayed

2.2 Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Reliability	Incomplete cases must not be finalized without clarification	Missing-field validation before routing
Maintainability	Modules should be separated clearly	Frontend / backend / AI adapter / validation / DB separation
Performance	Workflow response should be fast enough for live demo use	Short prompts, lightweight API design
Cost Efficiency	Token usage should remain controlled	Relevant context only, structured prompt design
Traceability	Every case should have visible state and stored log	Case + workflow_log tables

2.3 Out of Scope / Future Enhancements

The following items are not included in the preliminary-round MVP:

- Full ERP / POS integration
- Supplier portal integration
- Enterprise analytics dashboard
- Advanced OCR for poor-quality scanned handwritten invoices
- Multi-level production approval workflow
- Automated external supplier messaging

4.0 Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

StockFlow AI will be technically evaluated based on whether it can reliably convert unstructured discrepancy-related retail inputs into structured, actionable workflow cases. The evaluation focuses on the system's ability to perform case classification, field extraction, missing-information detection, workflow recommendation, and case state tracking in a way that demonstrates clear architecture, multi-step reasoning, and adaptive workflow behaviour. This is aligned with the domain requirement that the system function as a stateful AI workflow engine rather than a simple chatbot.

4.1.1 Grayscale Rollout & A/B Testing

A lightweight technical evaluation approach will be used instead of a full production rollout. The system will be evaluated through controlled demo scenarios using predefined discrepancy cases. Where relevant, two flows variants may be compared:

1. **Manual / baseline flow** – user inputs are reviewed without AI assistance
2. **AI-assisted flow** – the system uses ilmu-glm-5.1 to classify the case, extract fields, detect missing information, and recommend next actions

4.1.2 Emergency Rollback (ER) & Golden Release

Since the system depends on an external LLM endpoint, a rollback strategy is necessary. If the AI service returns unusable, off-topic, malformed, or low-

confidence output, the system will not finalize the case automatically. Instead, it will:

- Keep the case in a safe state such as `PENDING_INFO` or `REVIEW_REQUIRED`
- Request clarification from the user
- Allow manual review using the latest stable workflow logic

The “**golden release**” for the prototype is the last stable version of the application that can still accept discrepancy input, store cases, and display manual review states even when AI output is not accepted. This approach is consistent with the sample SAD structure and the QA expectation that fallback and hallucination handling should be documented.

4.1.3 Priority Matrix

The following priority matrix will be used to assess technical issues during testing and refinement:

- **P1 Critical** – AI output cannot be parsed, the case cannot be created, or the workflow fails completely. Immediate fix required.
- **P2 High** – case classification is incorrect, important fields are extracted wrongly, or the system routes the case to the wrong next step. Must be fixed before demo.
- **P3 Medium** – clarification prompts are unclear, case summary quality is weak, or workflow status updates are not fully informative. Fix if time permits.
- **P4 Low** – UI layout issues, wording improvements, or minor formatting inconsistencies that do not affect workflow correctness

4.2 Monitoring

For the MVP, the system will monitor:

- Case creation success
- Output schema validity
- Missing-field detection behaviour
- State transition correctness
- And LLM response success/failure

4.3 Assumptions

- Users can provide at least a short text description of the issue

- Some invoice or stock-related information is available in text form
- The prototype is evaluated as a workflow prototype rather than full production software
- One primary workflow is sufficient for preliminary round demonstration

4.4 External Dependencies

Tool / Dependency	Purpose	Risk
Ilmu-glm-5.1 model	Core reasoning engine for classification, extraction, missing-information detection, and structured output	High – if unavailable, core AI workflow capability is affected
FastAPI	Backend API framework	Low
Streamlit	Demo frontend	Low
SQLite	Case persistence	Low

5.0 Project Management & Team Contributions

5.1 Project Timeline

Days	Work Distributed
Day 1-2	Problem framing, PRD, architecture design
Day 3-4	Backend and workflow controller setup
Day 5-6	Frontend and database setup
Day 7-8	LLM integration and prompt refinement
Day 9	Testing and documentation finalization
Day 10	Pitch deck, demo polish, and video recording

5.2 Team Members Role

Member 1 (Hiew Yi Hwa) – Backend, Frontend, AI, LLM integration, QA testing, Documentation, Pitch deck and video lead

5.3 Recommendations

To maximize clarity and scoring potential:

- Keep the MVP focused on a single workflow only
- Make the LLM service visibly central in both architecture and demo
- Show at least one happy path and one missing information / ambiguity path
- Keep outputs structured and easy for judges to inspect
- Finalize diagrams so they match the actual implemented flow

6.0 Reference

FastAPI. (n.d.). FastAPI. <https://fastapi.tiangolo.com/>

ILMU API. (n.d.). Overview. <https://docs.ilmu.ai/docs/getting-started/overview>

ILMU API. (n.d.). Usage & cost. <https://docs.ilmu.ai/docs/admin/usage-cost>

Microsoft. (n.d.). Microsoft 365 Copilot [Large language model].
<https://www.microsoft.com/en-us/microsoft-365-copilot>

Python Software Foundation. (2026). Python 3.14.4 documentation.
<https://docs.python.org/3/index.html>

SQLite. (2025, November 13). About SQLite. <https://sqlite.org/about.html>

Streamlit. (n.d.). Streamlit documentation. <https://docs.streamlit.io/>

Visual Studio Code. (n.d.). Visual Studio Code. <https://code.visualstudio.com/>

Z.AI. (n.d.). GLM-5.1 overview. <https://docs.z.ai/guides/llm/glm-5.1>